K-Fold Cross-Validation, Worked Out by Hand

Fold splits, per-fold scores, and mean ± std aggregation

A complete numerical walk-through of one 5-fold cross-validation on a tiny dataset of $n = 10$ samples. We build the five fold index sets, run the five train/validate rounds, collect five validation accuracies, and then aggregate them into a single reliability estimate: the mean, the sample standard deviation, and the min/max envelope. Nothing is hand-waved — every number below was produced by the NumPy reference program in Appendix A, so the arithmetic is exact and self-consistent.

What this document does. It fixes a concrete tiny problem ($n = 10$, $k = 5$), lays out the exact index sets each fold trains and validates on, takes five plausible per-fold validation accuracies $(0.80, 0.85, 0.75, 0.90, 0.82)$, and derives by hand the aggregate score 0.824 ± 0.056 together with the [0.75, 0.90] range that quantifies how much the estimate wobbles across folds.

How to read this. Each step is self-contained and several carry a coloured box that isolates the idea. The mechanics (Sections 1–3) are pure bookkeeping; the statistics (Section 4) are one mean and one corrected variance; the stratified variant (Section 5) is the same recipe with a class-balance constraint. Read this to understand why CV returns a number *with an uncertainty* instead of one fragile point estimate.

Concepts covered, in order: the single-split problem · partitioning n samples into k disjoint folds · the k train/validate rounds and their exact index sets · collecting per-fold scores · the mean · sample (corrected) standard deviation · reporting mean ± std and min/max · stratified k-fold on an imbalanced set · when k-fold is the right tool versus a single split or grouped/temporal CV · the leakage trap.

1 The setup: why one split is not enough

You have a small labelled dataset and you want to know how well your model will generalise. The naive recipe holds out a single validation slice — say the last 2 of 10 samples — trains on the other 8, and reports that one accuracy. The problem is that this number is *fragile*: it depends entirely on *which* 2 samples landed in the holdout. A lucky split flatters the model; an unlucky one condemns it. On small data the swing is large, and you have no way to see it from a single number.

k-fold cross-validation removes the luck of the draw by rotating the holdout. Partition the data into k equal disjoint *folds*; then run k rounds, each time validating on a *different* fold and training on the remaining $k-1$. Every sample is validated on exactly once, and every sample is trained on in exactly $k-1$ of the rounds. You end up with k scores instead of one, and their spread *is* the uncertainty.

Quantity	Symbol	Value here
Number of samples	n	10
Number of folds	k	5
Fold size	n/k	$10/5 = 2$
Train size per round	$n - n/k$	8
Times each sample is validated	—	1
Times each sample is trained on	$k-1$	4

Intuition

A single train/validation split asks one question once: "how did the model do on *this* particular holdout?" Cross-validation asks the same question k times with k different holdouts and reports both the *average* answer and how much the answer *moved*. It reuses limited data to estimate generalisation *with uncertainty* — something a single fragile split can never give you. The mean tells you the typical performance; the spread tells you how much to trust the mean.

2 Step 1 — Partition the 10 samples into 5 folds

Label the samples $0, 1, \ldots, 9$. With $k = 5$ folds of size 2, the contiguous partition assigns the samples in order (in practice you shuffle first; here we keep them ordered so every index is trivially checkable):

$$F_0 = \{0,1\}, \quad F_1 = \{2,3\}, \quad F_2 = \{4,5\}, \quad F_3 = \{6,7\}, \quad F_4 = \{8,9\}$$

The folds are *disjoint* (no sample appears twice) and *exhaustive* (their union is all of $\{0, \ldots, 9\}$). That is the entire requirement for a valid k-fold partition:

$$F_i \cap F_j = \varnothing \ (i \neq j), \qquad \bigcup_{i=0}^{4} F_i = \{0, 1, \ldots, 9\}.$$

Mini-example — this operation in isolation

The fold size need not divide n evenly. For $n = 11$, $k = 5$ a standard splitter (`np.array_split`) makes folds of sizes $3, 2, 2, 2, 2$ — the first $n \bmod k = 1$ fold absorbs the remainder. Every sample is still validated exactly once; the only change is that two folds differ in size by one. Here $10 = 5 \times 2$ divides evenly, so all five folds have size 2 and the bookkeeping is perfectly symmetric.

3 Step 2 — The five train/validate rounds

In round i the validation set is fold F_i and the training set is everything else, $\text{Train}_i = \{0, \ldots, 9\} \setminus F_i$. Writing all five rounds out:

Round i	Validate on F_i	Train on the other 8	$\lvert\text{train}\rvert$
1	$\{0, 1\}$	$\{2, 3, 4, 5, 6, 7, 8, 9\}$	8
2	$\{2, 3\}$	$\{0, 1, 4, 5, 6, 7, 8, 9\}$	8
3	$\{4, 5\}$	$\{0, 1, 2, 3, 6, 7, 8, 9\}$	8
4	$\{6, 7\}$	$\{0, 1, 2, 3, 4, 5, 8, 9\}$	8
5	$\{8, 9\}$	$\{0, 1, 2, 3, 4, 5, 6, 7\}$	8

Read the table column-wise to confirm the two invariants. Down the "Validate" column each sample appears exactly once: sample 4 is held out only in round 3. Down the "Train" column each sample appears in exactly $k - 1 = 4$ rounds: sample 4 is trained on in rounds $1, 2, 4, 5$. The model in each round is trained *from scratch* — fresh weights, fresh optimiser — so the five validation scores are five independent looks at generalisation.

Each row is one round; the amber cells are that round's validation fold, the green cells its training set. The amber cells march diagonally — that diagonal is exactly what "rotate the holdout" means.

Watch out

The five models must be trained *independently*. A common bug is to keep training one model across folds (warm-starting from the previous fold's weights). Then the model has already seen fold F_2's data while it is "validating" on it later, and the scores are optimistically biased. Re-initialise the model and optimiser at the top of every round.

4 Step 3 — Per-fold scores and the aggregate estimate

Each round produces one validation accuracy. Suppose the five rounds returned:

$$a_1 = 0.80, \quad a_2 = 0.85, \quad a_3 = 0.75, \quad a_4 = 0.90, \quad a_5 = 0.82.$$

These are the raw material; aggregation turns them into one reliability estimate.

The mean. The headline CV score is the arithmetic mean of the per-fold scores:

$$\bar{a} \;=\; \frac{1}{k}\sum_{i=1}^{k} a_i \;=\; \frac{0.80 + 0.85 + 0.75 + 0.90 + 0.82}{5} \;=\; \frac{4.12}{5} \;=\; \boxed{0.824}.$$

The spread. A mean alone hides the wobble. We measure it with the *sample* standard deviation (Bessel-corrected, dividing by $k-1$, because the five folds are a sample and we estimated the mean from them). First the squared deviations from $\bar{a} = 0.824$:

Fold i	a_i	$a_i - \bar{a}$	$(a_i - \bar{a})^2$
1	0.80	−0.024	0.000576
2	0.85	0.026	0.000676
3	0.75	−0.074	0.005476
4	0.90	0.076	0.005776
5	0.82	−0.004	0.000016
		$\sum(a_i - \bar{a})^2 =$	0.012520

Divide by $k - 1 = 4$ to get the sample variance, then take the square root:

$$s^2 \;=\; \frac{1}{k-1}\sum_{i=1}^{k}(a_i - \bar{a})^2 \;=\; \frac{0.012520}{4} \;=\; 0.003130, \qquad s \;=\; \sqrt{0.003130} \;=\; \boxed{0.056}.$$

The envelope. Finally the raw extremes, which bound the worst and best fold:

$$\min_i a_i = 0.75, \qquad \max_i a_i = 0.90, \qquad \text{range} = 0.90 - 0.75 = 0.15.$$

Putting it together, the reliability estimate you report is

$$\boxed{\text{CV accuracy} \;=\; 0.824 \pm 0.056 \quad \text{(range 0.75–0.90)}}$$

Read as: the model generalises to roughly 82.4% accuracy, and the typical fold-to-fold swing is about ±5.6 points; even in the worst fold it did not drop below 75%. The ±0.056 band spans [0.768, 0.880] — that interval, not the bare 0.824, is the honest summary of what you know.

Intuition

Why divide by $k - 1$ and not k? Because $\bar{a}$ was itself computed from the same five numbers, the deviations $a_i - \bar{a}$ are slightly too small on average — the data hugs its own mean. Dividing by $k - 1$ instead of k inflates the estimate just enough to correct that bias. With only $k = 5$

folds the difference is real: dividing by 5 would give $s = 0.050$ instead of 0.056, understating the uncertainty by about 11%.

Watch out

Leakage is the silent killer of CV. Any data-dependent preprocessing — standardising features, imputing missing values, selecting features, fitting a target encoder — must be fit on the *training fold only* and then *applied* to the validation fold. If you scale or impute using statistics computed over the whole dataset *before* splitting, information from the validation fold leaks into training and every per-fold score is optimistically inflated. The fix: build the preprocessing inside the per-fold loop, after `train_idx`/`val_idx` are chosen, e.g. a scikit-learn `Pipeline` fit on the training fold.

5 Step 4 — Stratified k-fold for imbalanced classes

Plain k-fold splits at random with no regard for labels. On an *imbalanced* dataset that is dangerous: a rare class can be heavily over- or under-represented in a fold, or even vanish from one entirely, making that fold's score meaningless.

A small imbalanced set. Take $N = 20$ samples with 4 positives (class 1) and 16 negatives (class 0), so the overall positive rate is

$$\frac{4}{20} = 0.20 \quad (20\% \text{ positive}).$$

Run plain 2-fold CV and you might draw a fold of 10 samples containing 0, 1, or all 4 positives purely by chance — the positive rate in that fold could be anywhere from 0% to 40%, nothing like the true 20%.

Stratified k-fold fixes this by splitting *each class separately* and then combining, so every fold preserves the global class proportions. With 4 positives and 2 folds, it places 2 positives in each fold; with 16 negatives, 8 in each:

$$\text{Fold A: } 2 \text{ pos} + 8 \text{ neg}, \qquad \text{Fold B: } 2 \text{ pos} + 8 \text{ neg}.$$

Each fold's positive rate is now $2/10 = 0.20$ — exactly the global 20%. The estimate is no longer at the mercy of where the four rare samples happened to fall.

	Positives	Negatives	Positive rate
Whole set	4	16	0.20
Stratified fold A	2	8	0.20
Stratified fold B	2	8	0.20
Plain fold (unlucky)	0	10	0.00

When this is the right choice

Use **stratified** k-fold by default for classification, and *always* when classes are imbalanced or the dataset is small — it keeps every fold representative and shrinks the variance of the per-fold scores. Plain `KFold` is fine only for regression or large, well-balanced data. The cost is nil: stratification changes which indices land in each fold, not the number of folds or rounds.

6 Step 5 — When k-fold is the right tool

Cross-validation is not free — it trains k models instead of one — and it is not always the correct validation scheme. Match the method to the data:

When this is the right choice

Small data (n in the hundreds to low thousands): use k-fold. A single split wastes too much data on the holdout and gives a noisy estimate; rotating the holdout recovers both signal and an uncertainty bar.

Large data (tens of thousands or more): a single held-out split is usually enough — the holdout is large enough to be stable, and k-fold's $k\times$ training cost is not worth it.

Grouped data (multiple rows per patient, user, or device): use `GroupKFold` so all rows from one group stay on the same side of the split; random folds would leak group identity.

Temporal data (time series): *never* split randomly — it lets the model train on the future and validate on the past. Use `TimeSeriesSplit`, which always validates on a window that comes *after* the training window.

Watch out

Applying ordinary random k-fold to a time series is a classic, invisible failure: the reported CV score looks great, then the model collapses in production. The leakage is temporal — future information bleeds backward through the random shuffle. If your rows have a natural order or grouping, a plain shuffle is almost always wrong.

7 The whole procedure, at a glance

#	Step	For our run	Value
1	Choose k	5 folds of size 2	$k = 5$
2	Partition n	$\{0,1\}\ldots\{8,9\}$	5 disjoint folds
3	Run k rounds	validate F_i, train rest	8 train / 2 val each
4	Collect scores	a_i per fold	$0.80, 0.85, 0.75, 0.90, 0.82$
5	Mean	$\frac{1}{k}\sum a_i$	0.824
6	Sample std	$\sqrt{\frac{1}{k-1}\sum(a_i-\bar{a})^2}$	0.056
7	Min / max	worst / best fold	0.75/0.90
8	Report	mean $\pm$ std	0.824 ± 0.056

These eight rows *are* a cross-validation run. The first three are bookkeeping, the next two are one

mean and one corrected variance, and the last reports a number with its uncertainty instead of a single fragile point estimate.

8 Equation cheat-sheet

Fold size	n/k (remainder folds get one extra)
Valid partition	$F_i \cap F_j = \varnothing,\ \bigcup_i F_i = \{0, \ldots, n{-}1\}$
Round i	validate F_i, train $\{0, \ldots, n{-}1\} \setminus F_i$
Times trained / validated	$k-1$ / 1 per sample
CV mean	$\bar{a} = \frac{1}{k} \sum_{i=1}^{k} a_i$
Sample variance	$s^2 = \frac{1}{k-1} \sum_{i=1}^{k} (a_i - \bar{a})^2$
Sample std	$s = \sqrt{s^2}$
Report	$\bar{a} \pm s$ (plus $\min_i a_i,\ \max_i a_i$)
Stratified fold	class-c count per fold $= n_c/k$

Appendix A Reference implementation

Every number above is reproducible from the following program (NumPy; scikit-learn's `KFold`/`StratifiedKFold` produce the same index sets). Change `n`, `k`, or the accuracy list to explore.

```
import numpy as np

n, k = 10, 5

# --- Step 1: partition 0..9 into 5 folds of size 2 ---
idx   = np.arange(n)                # ordered; shuffle in practice
folds = np.array_split(idx, k)      # [array([0,1]), array([2,3]), ...]
for i, f in enumerate(folds):
    print(f"fold {i}: {f.tolist()}")

# --- Step 2: the k train/validate rounds ---
for i in range(k):
    val_idx   = folds[i].tolist()
    train_idx = np.concatenate([folds[j] for j in range(k) if j != i]).tolist()
    print(f"round {i+1}: val={val_idx}  train={train_idx}")
    # m = fresh_model(); train on train_idx; a_i = evaluate on val_idx

# --- Step 3: aggregate five per-fold accuracies ---
acc  = np.array([0.80, 0.85, 0.75, 0.90, 0.82], dtype=np.float64)
mean = acc.mean()                   # 0.824
std  = acc.std(ddof=1)              # 0.05595  -> 0.056  (Bessel: divide by k-1)
print(f"sum sq dev = {((acc-mean)**2).sum():.6f}")   # 0.012520
print(f"sample var = {acc.var(ddof=1):.6f}")         # 0.003130
print(f"CV accuracy = {mean:.3f} +/- {std:.3f}")     # 0.824 +/- 0.056
print(f"min={acc.min():.2f}  max={acc.max():.2f}  range={acc.ptp():.2f}")

# --- Step 4: stratified k-fold preserves class proportions ---
# 20 samples, 4 positive (class 1), 16 negative (class 0): rate = 0.20
y = np.array([1]*4 + [0]*16)
pos, neg = (y == 1).sum(), (y == 0).sum()
print(f"global positive rate = {pos / len(y):.2f}")  # 0.20
# split each class separately into k folds, then combine ->
# every fold has pos/k positives and neg/k negatives (rate preserved)
print(f"per stratified fold (k=2): {pos//2} pos + {neg//2} neg -> "
      f"rate {(pos//2)/(len(y)//2):.2f}")            # 0.20
```

— end of document —